

NOTA DE ESTUDO: Práticas de desenvolvimento de software e hardware confiáveis

Dentre as plataformas de software e hardware para desenvolvimento, se destacam as que se utilizam de componentes utilizados em sistemas reais, utilizados em produção. Sua utilização em um contexto real indica que a sua implementação é mais fidedigna com os padrões especificados e publicados pelo órgão padronizador.

Alguns destes ambientes permitem também que seja recriado um sistema de produção em um ambiente controlado, onde dispositivos físicos podem ser substituídos por dispositivos virtualizados. Estas plataformas reduzem a necessidade de equipamentos especiais, permitindo que diversas versões do sistema como todo sejam testados simultaneamente, de maneira contrastante com instalações físicas dedicadas para testes.

A seguir, são listadas as principais práticas para o desenvolvimento seguro de sistemas:

Utilização de linguagens de programação menos sujeitas a corrupção de memória

Segundo estudos sobre as vulnerabilidades e descobertas no software de grandes companhias, como Microsoft, Google, Mozilla e Apple são causadas devido a corrupção da memória de programas.

A corrupção de memória é o resultado de uma operação de leitura ou escrita da memória, que cause um estado inesperado ou inconsistente do programa. Existem diversos tipos de corrupção de memória, incluindo:

- uso de memória não inicializada, onde uma variável não inicializada pode conter um valor inesperado da memória deixada por outro programa executado anteriormente no mesmo espaço de memória física;
- invasão de memória adjacente, onde variáveis de memória localizadas de forma adjacente a variável alvo são acessadas indevidamente por erro do programador;
- uso após liberação de memória, onde um bloco de memória previamente alocado e já liberado é acessado indevidamente por meio de um ponteiro inválido;

- confusão de tipos, onde ponteiros para estruturas, objetos e funções são interpretados com os tipos incorretos, levando a comportamento inesperados;
- corrupção da pilha de programa (*stack*) ou da região dinâmica de memória (*heap*), através da alteração do fluxo de execução do programa ou estruturas em memória para gerência de memória dinâmica.

Tanto os compiladores de programa, quanto os sistemas operacionais e fabricantes de hardware oferecem mitigações para alguns destes problemas. Contudo, a proporção de vulnerabilidades causadas por estes tipos de falhas se manteve praticamente constante requerendo outro tipo de abordagem para sua completa eliminação.

A International Standards Organization (ISO) e International Electrotechnical Commission (IEC), através de seu Comitê Técnico Conjunto JTC1 dedicado à tecnologia da informação, criaram em 2006 o grupo de trabalho WG23 dedicado a documentação de falhas, vulnerabilidades decorrentes das falhas e as boas práticas para mitigá-las em diferentes linguagens de programação no ISO/IEC TR 24772. O padrão é recomendado para sistemas críticos, onde a corrupção de memória poderia levar ao funcionamento incorreto de equipamentos resultando em perda de vidas humanas, além de prejuízos ecológicos, logísticos e financeiros.

Mesmo seguindo padrões de boas práticas, mesmo programadores experientes podem cometer erros. Neste contexto, foram criadas uma série de ferramentas para checagem do código fonte e do programa em execução para a identificação destas falhas. A alternativa sugerida pela Agência de Segurança Nacional (NSA) dos Estados Unidos da América é a avaliação de componentes de software mais vulneráveis e a priorização da sua migração para linguagens de programação mais seguras.

Uma das linguagens seguras adotadas é chamada Rust, criada pela Mozilla Foundation. A Mozilla demonstrou o potencial de Rust reescrevendo um componente de seu navegador Firefox, o protegendo contra defeitos causados por corrupção de memória que correspondiam a 73.4% das vulnerabilidades reportadas para seu antecessor escrito na linguagem C++. No caso do sistema operacional Android, a migração de parte dos sistemas escritos nas linguagens C e C++ mais expostos à interação com usuários reduziu o número de

vulnerabilidades devido à corrupção de memória de 76% para 35% do total de vulnerabilidades reportadas.

Utilização de linguagens fortemente tipificadas

Segundo o Microsoft Security Response Center outra classe comum de falhas de implementação que tornam as aplicações vulneráveis é a confusão de tipos. A confusão de tipos se deve à falta de mecanismos de checagem durante a execução que verifiquem que um objeto em memória seja de fato o tipo de que se tenta operar. Quando o tipo requerido é diferente do tipo operado, o comportamento do programa é indefinido, podendo corromper a memória, gerando um estado inválido do programa e alterar seu fluxo de execução, gerando resultados incorretos.

O uso de linguagens fortemente tipificadas utiliza de checagem estática dos tipos esperados para prevenir que tipos incorretos sejam utilizados indevidamente. Porém, muitas destas mesmas linguagens como o C e C++ permitem a reinterpretação de um endereço de uma estrutura em memória, podendo acarretar no mesmo tipo de problema que uma linguagem fracamente tipificada. Além disto, o próprio compilador ou interpretador da linguagem pode introduzir falhas por confusão de tipos, que podem ser exploradas por ataques especulativos como Spectre, permitindo o roubo de dados confidenciais.

Alternativamente, os tipos podem ser checados durante o tempo de execução, impactando o desempenho da aplicação, mas garantindo que o programa não continue sendo executado quando o seu comportamento não for previsível.

Verificação léxica, sintática e semântica automática

A verificação consiste no processo de revisão em busca de falhas de projeto ou implementação que tenham passado despercebidas durante o processo de desenvolvimento, configuração e operação de software ou hardware. Em sistemas críticos, a verificação deve ser contínua, de preferência feita com automação.

A verificação pode ser feita em código fonte para software, ou linguagem de descrição de hardware para hardware, ou mesmo documentos de projeto definindo os componentes dos sistemas e como se organizam.

Existem três tipos de verificação. Na verificação léxica, são checadas se palavras e símbolos fazem parte de uma gramática correspondente. Um exemplo do tipo de falha léxica

é a utilização da palavra inglesa *word* em um texto em português. Em linguagens de programação, este tipo de falha pode ocorrer ao se utilizar um tipo ou símbolo não definido.

Na verificação sintática, são checadas se palavras e símbolos de uma gramática são utilizados de maneira correta. Um exemplo de falha sintática na língua portuguesa é o uso de um pronome errado, como na frase “os palavra”, onde o pronome “os” diverge em gênero e número do sujeito “palavra”. Em uma linguagem de programação, falhas sintáticas ocorrem quando tipos inadequados são utilizados, ou quando as afirmações de programa são malformadas.

Na verificação semântica são checadas se um conjunto de palavras e símbolos correspondem à expectativa da informação que se quer transmitir. Em linguagem natural, um exemplo de falha semântica seria afirmar algo absurdo, como “o céu é verde”. Já em linguagem de programação, seria a execução de um comando ilógico, que não produz resultado ou produz um resultado indesejado.

A checagem semântica automática, porém, é de difícil execução devido à sua necessidade de entendimento do contexto no qual um certo trecho de programa está inserido e que se deseja executar. A semântica é tipicamente checada manualmente através da etapa de verificação do programa, onde uma pessoa que não seja a desenvolvedora, lê o programa e procura por defeitos de implementação em termos de lógica. O avanço da aprendizagem de máquina com os Modelos Linguísticos Grandes (Large Language Models, ou LLM em inglês), como o famoso Chat-GPT, permitiu que também a checagem semântica do código seja feita de maneira automática e iterativa, já sendo utilizada experimentalmente para esta função na academia e indústria.

Verificação da presença de credenciais em imagens de software e repositórios

Uma das preocupações apresentadas é a presença de credenciais embarcadas nas imagens de software distribuídas e utilizada por operadoras de telecomunicações. O vazamento destas credenciais pode ser explorado por terceiros para ataques da infraestrutura de rede.

Mesmo companhias de desenvolvimento de software estabelecidas sofrem com este tipo de falha. Tipicamente devido à falta de atenção dos desenvolvedores e falta de sistemas de verificação da presença destas credenciais, como no caso recente onde as chaves privadas do GitHub foram inadvertidamente disponibilizadas para o público.

Utilizando a ferramenta de busca do CVE hospedado no GitHub podemos encontrar 474 CVEs reportadas relacionadas às credenciais em imagens de software (frase de busca “repo:CVEProject/cvelistV5 ((hard-coded) OR (hardcoded)) AND credential”). Os CVEs que foram classificadas com o CWE-798, relativo a este tipo de defeito, são 131 (string de busca “repo:CVEProject/cvelistV5 ((hard-coded) OR (hardcoded)) AND credential AND CWE-798”).

Segundo relatório da GitGuardian de 2021, mais de 5 (cinco) mil credenciais são vazadas por dia via repositórios de código públicos hospedados no GitHub. Mesmo credenciais corporativas são frequentemente vazadas a partir de repositórios pessoais de funcionários que criam suas cópias de trabalho e inadvertidamente as tornam públicas. Entre as organizações afetadas notórias estão o próprio GitHub, como reportado em, além do Uber, Nações Unidas e Equifax. Esta última, é uma empresa de avaliação de crédito responsável pelo vazamento dos dados pessoais de 140 milhões de americanos.

Dentre os tipos de credenciais vazadas, mostrados na Figura 1 se incluem chaves de API, credenciais de bancos de dados, aplicativos de comunicação, provedores de serviços de nuvem e chaves privadas, ou seja, todos os vazamentos graves que podem permitir ataques de personificação e roubo de dados.

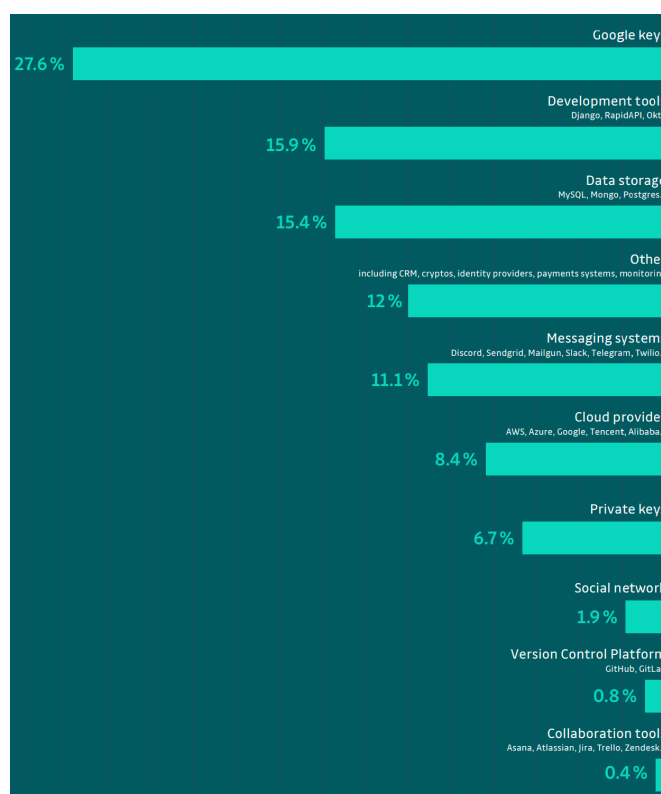


Figura 1. Tipos de credenciais vazadas via repositórios de código fonte públicos [132]

A verificação não exclui treinamento e as práticas seguras de trabalho, visto que funcionários são alvos preferenciais de ataques para roubo de credenciais. O roubo de credenciais pode permitir o acesso de terceiros à infraestrutura e serviços críticos, levando à interrupção dos serviços e capacidade de resposta do pessoal responsável. Um exemplo notório de roubo de credenciais aconteceu com o serviço SecurID da RSA, que permitiu a invasão da rede privada da fabricante de equipamentos para defesa Lockheed Martin, inclusive com tentativa de roubo de projetos militares. Segundo a investigação da própria RSA, um funcionário abriu uma planilha de dados que executou um script e instalou o malware explorando uma vulnerabilidade desconhecida, também chamada de *0-day*. Também foi descoberto que o ataque seria malsucedido se o software no computador do funcionário estivesse atualizado, reforçando a importância da atualização de softwares e capacitação dos funcionários.

Validação de integridade e autenticidade do software e hardware

A prática de validação de integridade e autenticidade de software e hardware também é recomendada em sistemas críticos, minimizando o potencial para ataques de cadeia de suprimentos. Neste tipo de ataque, o comprometimento de algum um dos fornecedores na cadeia de suprimento é capaz de comprometer a segurança do restante da cadeia, até alcançar o alvo do ataque.

Um exemplo deste tipo de ataque é o comprometimento do serviço SecurID da companhia RSA que foi utilizado para a obtenção de credenciais de um funcionário da companhia Lockheed Martin, que foi então utilizada para tentativa de roubo de projetos militares.

Para validar a integridade de software, podem ser utilizadas soluções criptográficas. São calculados números através de funções de hash, que são enviados pelo fornecedor do software separadamente do mesmo. Ao receber o software, o seu consumidor deve se utilizar da mesma função de hash para gerar um novo número e comparar com o produzido pelo fornecedor do software. Caso os números de hash gerados pelo fornecedor e pelo consumidor diverjam, o software ou o número de hash fornecido pelo fornecedor foram corrompidos. Uma nova cópia de ambos deve ser obtida e o processo de cálculo dos hashes repetido. Caso este processo falhe repetidas vezes, pode indicar o comprometimento de algum dos participantes ou da rede, requerendo investigação por profissionais capacitados.

Caso o software esteja íntegro, com os hashes do fornecedor e consumidor idênticos, a autenticidade do software deve ser checada. Isto é feito a partir do processo de assinatura digital do pacote de software e de todos os artefatos contidos neste mesmo pacote, incluindo imagens, bibliotecas, programas executáveis, documentos de texto, áudio, vídeo, dentre outros possíveis artefatos.

O consumidor deve verificar a assinatura eletrônica utilizando o processo padrão de verificação com a emissora do certificado. Caso o certificado da assinatura esteja expirado ou seja inválida, o software deve ser descartado.

Com as verificações acima, garante-se apenas que o software recebido seja de fato o software entregue, e que consta a assinatura do fornecedor. Porém, não garante que sejam artefatos originais, visto que credenciais roubadas do fornecedor podem ser utilizadas para forjar uma assinatura válida em um ou múltiplos componentes de software comprometidos.

É recomendado que o produtor, por sua vez, utilize ferramentas de auto checagem de integridade de seu próprio software. Este tipo de ferramenta causa a interrupção do programa caso alguns dos artefatos do pacote de software, que possa impactar seu funcionamento, seja modificado por um atacante de maneira sorrateira.

Além de checagens criptográficas, o software deve ser testado com um programa antivírus, ter sua funcionalidade esperada validada por meio de testes, e ser continuamente monitorado para comportamentos anormais durante sua operação.

A autenticidade de hardware é checada através de chaves gravadas em memória de apenas leitura do dispositivo. Dependendo da criticidade do setor, a autenticidade do hardware pode ser confirmada por meio da comparação de imagens de raios-x dos equipamentos. A validação do hardware é feita através de equipamentos de teste que atestam que o dispositivo se comporta como esperado. Dependendo da criticidade do setor, a validação é feita pelo fornecedor e pelo consumidor, impedindo que componentes defeituosos, falsos ou maliciosamente modificados sejam utilizados na produção.

A GSMA informa que a firma de consultoria Umlaut se deparou com diversos casos de software para sistemas de redes sendo utilizados em produção sem nenhum tipo de validação da legitimidade, integridade e teste de aceitação. Em muitos destes casos, o software também não contava com proteção contra modificações ou checagens de integridade, podendo ser facilmente modificado para introduzir vulnerabilidades no sistema com total desconhecimento dos responsáveis pela operação. Desta maneira, um ataque poderia

persistir por anos até que fosse detectado pelos operadores da rede, de maneira similar ao caso da Lockheed Martin.

Baseado no padrão CIP-010-3, da Companhia de Confiabilidade Elétrica Norte Americana (NERC) sobre controle de mudanças de configuração em infraestrutura críticas, foi produzido o diagrama mostrado na Figura 2 mostrando o ciclo ideal para a atualização de software em infraestrutura críticas, com a etapa de baixar o software, checar o *hash* criptográfico, em seguida checar a assinatura digital, testar o software em uma instalação para testes antes de implantar na produção. Em seguida, executar testes de aceitação para se verificar se o produto entregue pelo fornecedor corresponde com as expectativas do cliente. Durante a fase de testes que precede a implantação em produção, monitorar o comportamento. Caso o comportamento do componente seja dispare do esperado, investigar a causa e apenas após a confirmação de que é o comportamento esperado, prosseguir para a implantação. O padrão CIP-010-3 ainda prevê o monitoramento do sistema e a documentação a cada 35 dias corridos de diferenças observadas no desempenho dos sistemas e no histórico de acesso. Em amarelo, está o ciclo encontrado na auditoria feita pela Umlaut para a GSMA.



Figura 2. Ciclo de atualização de software crítico e ciclo encontrado em auditoria em operadoras

Testes de unidade, integração, sistema, aceitação e conformidade

Parte do ciclo de vida para desenvolvimento de software seguro recomendado pelo GSMA inclui etapas de testagem de estática e dinâmica. Estas tarefas são executadas por ferramentas de testagem externas ao projeto de software ou hardware, sendo também utilizadas em diferentes domínios de aplicação. A testagem estática também pode ser feita por humanos, no processo conhecido como de verificação por pares.

As tarefas que compõem a testagem dinâmica envolvem casos de teste do domínio de aplicação, por exemplo o domínio da telefonia móvel. Estes casos de testes são montados de maneira a validar que uma implementação é funcional e que atende os requisitos funcionais e técnicos especificados. Os requisitos técnicos são definidos a partir do projeto do sistema que atende aos requisitos funcionais.

Os casos de testes podem ser subdivididos em quatro grandes grupos. O primeiro grupo é composto pelos testes de unidades, responsável por testar o comportamento de cada componente de um sistema de maneira isolada. Neste tipo de teste, são checados tipicamente se entradas conhecidas produzem resultados esperados. Caso o componente venha a apresentar falhas devido a defeitos no futuro, não só o defeito é corrigido, mas devem ser acrescentados novos casos de teste para evitar que os mesmos defeitos sejam reintroduzidos.

O segundo grupo são os testes de integração, onde um subconjunto de componentes é testado simultaneamente, com o objetivo de se validar as interfaces entre os componentes as interações entre eles. Estes subconjuntos de componentes podem ser agregados de maneira a se compor um sistema ou subsistema.

O terceiro grupo são os testes de sistema, onde o sistema tem suas funcionalidades requeridas testadas de maneira fim-a-fim, ou seja, que o sistema como todo produz os resultados esperados para as possíveis interações com os usuários. O objetivo é verificar se o sistema opera como esperado quando se integram os diferentes subsistemas que o compõe.

O quarto grupo é composto pelos testes de aceitação ou conformidade. Tipicamente são testes de sistema, mas seu intuito não é identificar e prevenir o reaparecimento de defeitos, mas sim garantir que um produto entregue de fato atende suas especificações requeridas e critérios de qualidade. Este tipo de teste é feito nas indústrias tradicionais durante o recebimento de suprimentos de produção, que são testados e caso não estejam dentro das especificações, devolvidos para o fabricante. Também é feito em alguns segmentos da indústria digital, como para certificação de desempenho, protocolos e funcionalidades de software e hardware.

A Figura 3 mostra o Modelo V, separado em sua metade esquerda por etapas de definição e especificação de características de um produto e finalmente sua implementação por unidades. A sua metade direita mostra as etapas onde as unidades são integradas em subsistemas ou componentes, que são testados individualmente. Em seguida, são testados

conjuntos de subsistemas nos testes de integração. No fim da etapa de desenvolvimento, é testado o sistema completo, de maneira fim-a-fim, replicando interações com usuários finais. Após isto, são executados testes de conformidade para garantir que os requisitos exigidos por agências regulamentadoras são seguidos. Após os testes de conformidade, o consumidor do sistema executa sua própria bateria de testes de aceitação, garantindo que o produto recebido se comporta como esperado nos casos previstos pelo consumidor, similar à análise de produtos recebidos de setores tradicionais da indústria. Caso o produto falhe em qualquer uma das etapas de teste, o produto deve receber correções e recomeçar a série dos testes do início. Com a aprovação em todos os testes, o produto é considerado meramente aceitável, porém não podem ser feitas mais afirmações a respeito de sua qualidade sem um estudo mais aprofundado.

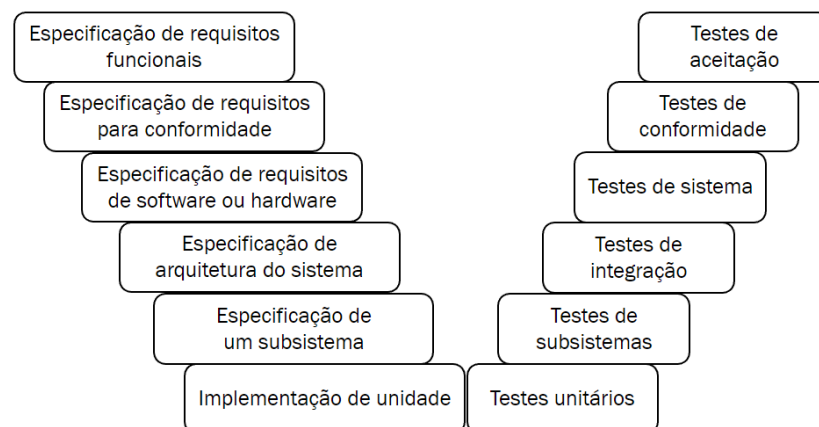


Figura 3. Modelo V

Configurações seguras por padrão

Conforme a complexidade dos sistemas crescem, cresce também a necessidade de parâmetros de configuração adicionais. Porém, certos parâmetros legados ou combinações de parâmetros podem tornar o sistema mais suscetível a ataques.

A GSMA recomenda, como parte do ciclo de vida para desenvolvimento de software seguro, que hardware e software venham configurados por padrão utilizando parâmetros de configurações conhecidamente seguras. Por si só, esta política de configurações seguras reduz a possibilidade de falha do operador, seja ela por falta de treinamento, falta de documentação do sistema, defeito na implementação do sistema ou defeito na especificação dos protocolos.

Dois exemplos de incidentes causados por configurações inseguras incluem o postergamento de atualizações automáticas do sistema operacional, que combinada com a não desativação da capacidade de execução de macros automáticas pelo Microsoft Excel, permitiu que uma vulnerabilidade desconhecida fosse explorada.

Outro exemplo recente inclui a invasão da Colonial Pipeline, companhia gestora de um dos maiores oleodutos e gasodutos nos Estados Unidos, provendo 45% (quarenta e cinco por cento) dos combustíveis consumidos pela costa leste, que levou a interrupção preventiva da planta por alguns dias. A invasão ocorreu devido ao reuso de credenciais de um funcionário em serviços diversos, que por sua vez foram atacados e tiveram vazamento destas credenciais reutilizadas. Com as credenciais em mãos, os atacantes acessaram a rede virtual privada (VPN) da companhia e disseminaram *ransomware* na rede interna, que poderia ter sido protegida através da autenticação de múltiplos fatores e da verificação da localização do endereço de IP utilizado durante o acesso. A invasão se traduziu em um esforço de outros prestadores de serviços críticos e do governo americano para impedir outra ocorrência similar devido aos transtornos.

O banco de controle de alterações em sistemas críticos, segundo a Companhia de Confiabilidade Elétrica Norte Americana (NERC) define em seu padrão CIP-010-3 devem ser seguidos a seguinte série de procedimentos:

- Levantamento de todas as configurações atualmente utilizadas em
 - Sistemas operacionais
 - Firmwares
 - Software de terceiros (comercial ou de código aberto) intencionalmente instalados
 - Softwares personalizados instalados
 - Portas lógicas da rede acessíveis
 - Lista de correções de segurança instaladas
 - Uma tabela identificando itens e configurações, de maneira e individualmente ou em grupo
 - Um sistema de rastreamento de equipamentos e configurações, de maneira individual ou em grupo
- Autorizar e documentar periodicamente as mudanças que desviarem das configurações de base levantadas na etapa inicial

- As requisições de modificações devem ser documentadas e devem receber autorização eletrônica de um indivíduo responsável ou grupo com autoridade para realizar a mudança
 - O indivíduo ou grupo que autoriza o pedido deve verificar antes de autorizar a modificação, que os controles de cibersegurança podem ser impactados pela mudança
 - Antes de implementar a mudança em produção, a configuração deve ser testada em ambiente controlado, que deve ser testado e monitorado para efeitos adversos
 - Após os testes e monitoramento, os resultados devem ser documentados antes da implementação no sistema de produção
 - Documentar que a mudança foi executada de acordo com o requerimento
- Atualizar o registro da configuração de base em no máximo 30 dias corridos da implementação de uma modificação de configuração
 - Monitorar continuamente e documentar a cada 35 dias corridos mudanças não documentadas da configuração de base, documentando também modificações não autorizadas e seu autor

Integração contínua

Uma das práticas que mais cresceram em popularidade nos últimos anos é o de integração contínua. Esta prática se trata do estabelecimento de um conjunto de operações utilizadas para construir, configurar e testar uma aplicação em uma cópia de um ambiente virgem.

A prática é importante pois permite que sejam encontrados problemas na disponibilidade das dependências de software necessárias em cada um desses ambientes com diferentes sistemas operacionais. Também permite que sejam encontradas incompatibilidades entre diferentes versões das dependências, que podem ser tratados antes da publicação do software para os seus usuários. Além disto, permitem a testagem em sistemas distintos, que podem apresentar diferentes comportamentos que precisam ser tratados corretamente. Posterior aos testes, se pode realizar os procedimentos de encapsulamento do software para distribuição, assim como serão consumidos por seus consumidores finais. Por fim, todas as ferramentas de checagens anteriormente vistas podem

fazer parte de uma sequência de tarefas que compõe o fluxo de integração contínua, também chamado de *pipeline* em inglês.

Em resumo, a integração contínua permite que todo o conjunto de desenvolvedores e a gerência acompanhe problemas identificados automaticamente, a frequência com que estes problemas ocorrem e o quão rapidamente são consertados. Isto permite a decisão informada de quando é possível publicar uma nova versão do produto que está sendo trabalhado.

A integração contínua permite a prática de entrega contínua (*continuous delivery* ou CI em inglês), onde o software é publicado automaticamente para os usuários se o fluxo de integração contínua (*continuous delivery* ou CD em inglês) contendo checagem, construção e testagem do software e seu encapsulamento, seja finalizado com sucesso. As duas práticas associadas são chamadas de CI/CD no contexto de “Desenvolvimento e Operações” (DevOps em inglês).

Entrega contínua, aprendizado de máquina e operações de segurança

A prática de integração contínua inspirou diversos segmentos tradicionais da indústria, como a indústria automotiva, a adotarem práticas semelhantes adaptadas ao seu contexto.

A indústria automotiva tradicional, por exemplo, conta com um ciclo de retroalimentação de segurança, onde diferentes eventos que ocorrem após o início da produção retroalimentam etapas de desenvolvimento. Esta retroalimentação pode resultar na alteração do calendário de manutenção preventiva de um dado componente, ou no completo redesenho do componente defeituoso, ou até mesmo o redesenho do produto como todo, dependendo da frequência e gravidade de problemas, incidentes e acidentes ocorridos devido à um componente específico. Os diferentes ciclos de retroalimentação de segurança são mostrados na Figura 4 onde estão presentes:

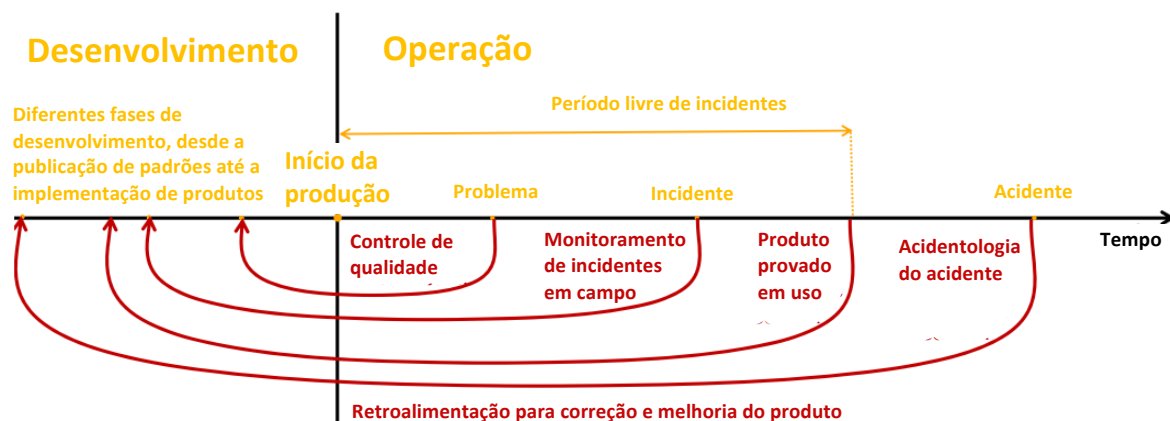


Figura 4. Ciclos de Retroalimentação para Controle de Confiabilidade de Produtos Críticos

Este ciclo, porém, não leva em conta a nova disponibilidade de carros conectados as redes 4G e 5G, que lhes permite acesso atualizações instaladas remotamente (*over-the-air*, em inglês), invés de apenas em oficinas. Isto permite que os sistemas embarcados sejam corrigidos para mitigar problemas via software ou mudar parâmetros de avisos de problemas mecânicos, quando possível.

Além dos aspectos de manutenção, outro segmento que vem crescendo em importância e popularidade é o de carros guiados através de sistemas inteligentes. Estes sistemas tipicamente são baseados em aprendizagem de máquina, onde dados de sensores diversos são coletados, processados e decisões de controle, como mudar de faixa de rolamento, acelerar e desacelerar o veículo são tomadas.

Os modelos embarcados de inteligência artificial, porém, ainda são imaturos quando comparados como seres humanos, que são mais adaptáveis a casos inusitados (não treinados ou infrequentes o suficiente para não gerarem uma resposta adequada). Para mitigar estes riscos, estes sistemas tipicamente contam com dois subsistemas completamente independentes, onde um subsistema fica ativo, enquanto o outro em fica em espera. O subsistema em espera age como um copiloto em treinamento, tomando decisões simultaneamente com o subsistema operante. As decisões de ambos são posteriormente transmitidas de volta para os servidores de suas fabricantes e são comparados em termos de eficiência, acurácia e segurança.

Este esquema de dois sistemas ativos simultaneamente é chamado de implantação Azul/Verde (*blue/green deployment*, em inglês) no contexto de DevOps, e faz parte do processo chamado de entrega contínua (*continuous-delivery*). Nele, quando o copiloto (Verde) assume a maturidade através de decisões corretas durante seu período supervisionado (por azul, simulações e operadores humanos), ele assume o controle (sendo chamado de azul), e o antigo piloto é substituído por outro sistema em período de aprendizagem supervisionada.

Exercícios

1. Explique o que é corrupção de memória e cite três exemplos de falhas relacionadas a esse problema descritas no texto.
2. Segundo o texto, por que linguagens de programação como Rust são consideradas mais seguras do que C e C++ em relação à corrupção de memória?

3. Diferencie verificação léxica, sintática e semântica, apresentando um exemplo de cada uma citados no documento.
4. Quais são os riscos associados ao vazamento de credenciais em repositórios públicos de código fonte e quais organizações foram citadas como afetadas por esse problema?
5. Descreva como ocorre a validação de integridade e autenticidade de software utilizando funções de hash e assinaturas digitais.
6. Explique a diferença entre testes de unidade, testes de integração, testes de sistema e testes de aceitação no desenvolvimento seguro de software.